LLM Notes: How LLMs and Tool Use Work

Oct 25, 2026 · @Pargat Singh

1. What an LLM is

An LLM is two things: a weights file and a small program that runs it.

- **Weights file**: billions of numbers (Llama 2 70B = 70 billion numbers, about 140 GB). No text or sentences are stored inside. The knowledge lives in the values of the numbers.
- **Small program**: a few hundred lines of code that loads the weights and does the math to predict the next word.

Think of it like a generic interpreter plus a huge config of numbers. The program is simple. The weights are what make it smart, and producing them is what costs millions.

2. How it runs (inference)

The LLM produces one word (token) at a time, in a loop.

1. The program turns your text into numbers (tokens).
2. It runs those numbers through the math using the billions of weights.
3. Out comes a probability for every possible next word, e.g. "Paris" 92%, "Lyon" 3%.
4. It picks one, adds it to the text, and goes back to step 2 until the answer is done.

It does not always pick the highest probability. Usually it samples: picks randomly, weighted by the probabilities. That is why the same question can get slightly different answers.

- **Greedy decoding** = always pick the top word (predictable, can be repetitive).
- **Temperature** = how much randomness is allowed. Low = safe and consistent. High = creative and random.

Hallucination: when the model doesn't know something, it still predicts words that sound right. It is confident autocomplete with nothing behind it.

3. Pretraining vs finetuning

Pretraining gives the model knowledge. Finetuning gives it behavior. Both use the same task: predict the next word.

	Pretraining	Finetuning
Data	~10 TB of internet text (huge, messy)	~100K human-written Q&A conversations (small, high quality)
Result	**Base model**: knows a lot, but only continues text	**Assistant model**: knows a lot and answers helpfully
Gives the model	Knowledge	Format and behavior (incl. when to write tool requests)
Cost	Millions of dollars, thousands of GPUs	Cheap, about a day

Base model problem: ask "What's the capital of France?" and it may reply with more questions, as if continuing a quiz page.

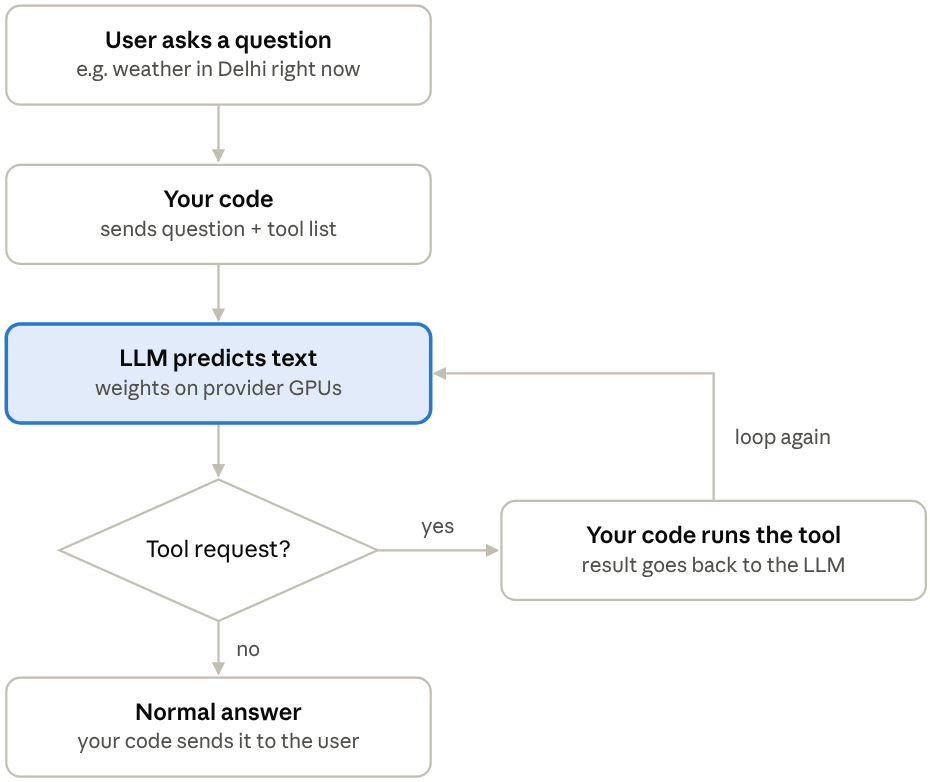
Analogy: pretraining = reading every book in a library. Finetuning = a short course on how to work a help desk.

Finetuning does not fix hallucination. It only steers the output toward helpful-assistant answers.

4. Tool use: how the LLM gets real-world info

The LLM can only write text. When it needs something it doesn't know, it writes a tool request, and your code does the actual work.

The LLM only predicts text; your code does everything else



tool use loop · your code vs the LLM

Read it top to bottom. A tool request sends the result back to the LLM for another round; a normal answer ends the loop.

The steps

1. User asks a question → your code receives it.
2. Your code sends the question + the tool list to the LLM, as plain text over an API.
3. The LLM predicts text: either a normal answer or a tool request.
4. Your code checks the format with an if/else:
   - Normal answer → send it to the user. Done.
   - Tool request → run the function (e.g. internet search).
5. Your code sends the result back to the LLM → back to step 3.

The LLM can make several tool requests in a row before giving the final answer.

What a tool request looks like

```
{ "tool": "search_internet", "input": "Denver weather today" }
```

It is just predicted text in a fixed format, like a request body you'd send to an API: which function, with what input.

Scenario 1: "What is the capital of India?"

Your code sends the question + tool list. The LLM already knows this from pretraining, so it predicts "New Delhi" directly. Your code sees a normal answer and sends it to the user. One LLM call, no tool used.

Scenario 2: "What is the exact weather in Delhi right now?"

Your code sends the question + tool list. The LLM's first prediction is a tool request for the weather tool. Your code runs the function, gets the temperature, and sends it back. The LLM writes the final answer, and your code sends it to the user. Two LLM calls, one tool used.

Things I got confused about

- **The code doesn't know in advance if a tool is needed.** It always sends the tool list; the LLM decides.
- **The LLM never runs the tool or touches the internet.** Your code does, then hands the result back.
- **The LLM doesn't "check" if it knows the answer.** It just predicts. Finetuning taught it to predict a tool request when the info is missing.
- **Your code only checks the format** (tool request vs answer), not whether the answer is good.
- **The tool list is written by the developer**, or generated by frameworks like LangGraph from your Python functions.

Practice script: `tool_use_demo.py` runs both scenarios and prints each round.

5. Where everything lives + key terms

There are two separate programs. Your code and the LLM only exchange text over an API.

	Where it lives	What it does
LLM (weights + its run program)	Provider's servers (Anthropic, Google, OpenAI) on their GPUs	Turns text into tokens, runs the weights, predicts the next words
Your code (e.g. tool_use_demo.py)	Your laptop / backend server	Sends question + tool list, checks the reply, runs the tools, decides when to stop

Key terms

- **Token**: a piece of text turned into a number the model works with.
- **Base model**: output of pretraining; continues text only.
- **Assistant model**: base model after finetuning; answers questions.
- **Tool**: a normal function in your code (search, database query, calculator).
- **Tool list**: descriptions of your tools that you write and send with every LLM call.
- **Tool request**: structured text the LLM predicts, meaning "run function X with input Y".
- **Agent loop**: your code calling the LLM repeatedly until it gives a normal answer.
- **Temperature**: how much randomness is allowed when picking the next word.
- **Hallucination**: confident, plausible-sounding text with no real knowledge behind it.